PharmaSoft - Actividad autónoma S9

José Condori Peña · Lenguaje de Programación II

C1. Resumen de ejecución

Se verificaron los 17 casos base y cuatro casos propios, cada uno de un grupo distinto. La comprobación combina la SPA, pruebas automatizadas de componentes y peticiones HTTP reales. Se prepararon tres clientes activos, uno inactivo y ocho productos activos; A4 comenzó con stock 2 y A2 tiene precio S/ 18.90. Las pruebas nuevas se ejecutaron con H2 temporal, API 8082 y SPA 4202, sin modificar Oracle.

Grupo	Ejecutados	Pasa	Falla
Buscadores	3	3	0
Cálculos	4	4	0
Validaciones	6	6	0
Confirmación	3	3	0
Mensajes	3	3	0
Impresión	1	0	1
Consistencia	1	1	0
Total	21	20	1

Resultado de regresión: 40 pruebas del frontend y 11 del backend aprobadas. La compilación de producción del frontend terminó correctamente. Los 13 escenarios HTTP adicionales también pasan después de la corrección. La fila I-01 se mantiene en Falla documental: las capturas acreditan el formato, pero faltan los tres archivos PDF nativos pedidos por la guía.

Los JSON y logs se identifican como tales. No equivalen a capturas de F12 o Postman. V-05 y M-02 se ejecutaron mediante HTTP real y tienen colección reproducible; queda conservar su ejecución en Postman. F-02 y P-03 cuentan con comprobación automatizada. La matriz y cada evidencia indican el modo de ejecución.

C2. Hallazgos y correcciones

H-01. La API truncaba cantidades decimales - alta - corregido

Caso asociado: V-02 y API-V02-2.5. Al enviar cantidad 2.5, el resultado inicial fue HTTP 201, total S/ 25.00 y descuento de dos unidades. Se esperaba HTTP 400 sin registrar ni alterar stock. Evidencia: hallazgo-H01-antes.json. La causa es la conversión predeterminada de un número decimal a Integer antes de validar DetalleVentaRequestDTO.cantidad; @Min no detecta la fracción ya descartada.

Corrección en la capa de entrada del backend: src/main/resources/application.yaml configura spring.jackson.deserialization.accept-float-as-int: false. VentasIntegrationTest.rechazaCantidadDecimalSinDescontarStock comprueba HTTP 400 y stock sin cambios. El reensayo real API-V02-2.5_api.json confirma el rechazo. No se intenta resolver esta regla únicamente en el formulario.

H-02. Aviso vacío dependiente de un valor no reactivo - media - corregido

La prueba P-01 detectó que la búsqueda sin coincidencias no aparecía al actualizar el control desde una prueba de componente. La plantilla leía buscarCliente.value para decidir si mostrar los resultados. VentaForm conserva ahora textoCliente y textoProducto como señales accesibles; venta-form.html utiliza textoCliente().trim() y textoProducto().trim(). La regresión y P-01_spa.png confirman el aviso. La corrección corresponde a presentación/reactividad del frontend.

H-03. Archivos de impresión incompletos - baja - pendiente

Caso I-01. Las tres capturas previas muestran comprobante con DNI y pie, sello ANULADA y consulta con filtros. Sin embargo, no están disponibles los tres PDF nativos guardados desde el navegador. Es un pendiente de evidencia, no un fallo demostrado de impresión. Corrección propuesta: imprimir cada vista con Guardar como PDF y conservar I-01-comprobante.pdf, I-01-anulada.pdf e I-01-consulta.pdf en docs/evidencias-s9/.

C3. Análisis de la solución

1. Por qué el total es estimado

VentaForm.total calcula una referencia a partir del catálogo cargado y redondea los importes. Entre esa consulta y la confirmación pueden cambiar los precios o el stock. VentaServiceImpl.registrar obtiene los precios vigentes y calcula el importe definitivo dentro de la transacción; VentaDetalle muestra la respuesta guardada. Si el backend aceptara el total del navegador, una petición modificada podría registrar una venta por un monto arbitrario, además de generar inconsistencias contables. Por eso el request contiene identificadores y cantidades, sin precios ni total.

2. Producto repetido en dos líneas

La regla adoptada en V-05 es rechazar los identificadores repetidos con 409. VentaForm.agregarProducto incrementa la línea existente, y VentaServiceImpl.registrar valida la unicidad antes de descontar. Así el comprobante contiene una sola línea por producto y el stock se descuenta una vez por su cantidad acumulada. Aceptar duplicados sería posible si el servidor los consolidara expresamente antes de validar; procesarlos sin esa regla hace menos claro el comprobante y complica el control del stock. La API real devuelve el 409 esperado.

3. Recuperación después del conflicto de stock

El cajero debe conservar cliente y detalle, conocer el stock actualizado y corregir solo la línea afectada. VentaForm.confirmar abandona el modo de guardado y confirmación cuando recibe 409; conserva el formulario, muestra el mensaje y actualiza el catálogo. La captura M-01 muestra cantidad 2 conservada y stock disponible 0. Como siguiente mejora, el mensaje podría enfocar automáticamente esa línea y ofrecer retirarla o ajustar su cantidad a las unidades disponibles, con confirmación del usuario antes de continuar.

4. Lista sin paginación en el reporte

El reporte agrupa por categoría: su cardinalidad depende del catálogo de categorías, que suele ser pequeño, y no del número histórico de ventas. Recibir todas las filas permite calcular con computed las unidades, el monto general y el mayor importe para barras comparables. ReporteCategorias.porcentaje calcula monto/mayor×100 y devuelve cero si no hay importes. La consulta de ventas, en cambio, crece por operación y usa paginación del servidor para no transferir todo el historial. Si el catálogo tuviera miles de categorías, habría que revisar esta decisión.

Matriz de pruebas ejecutadas

Cada ficha conserva las columnas exigidas por la guía. Pasa significa que el resultado observado coincide con el esperado en la modalidad indicada; no certifica que todas las capturas formales estén completas.

B-01 · Buscadores · Pasa

Precondición: 3 clientes activos, 1 inactivo

Datos: DNI 81000003 y apellido huaman

Esperado: Inactivo no aparece; Ana Huamán sí

Obtenido: Aviso vacío para el DNI; aparece Ana sin exigir tilde

Modalidad: SPA + prueba automatizada

Evidencia: B-01-inactivo_spa.png; B-01_spa.png

B-02 · Buscadores · Pasa

Precondición: 8 productos activos iniciales

Datos: Buscar A1 y analgesicos; Enter

Esperado: Precio/stock visibles; Enter agrega primero sin resumen

Obtenido: A1 encontrado por nombre y categoría; una línea creada

Modalidad: SPA + prueba automatizada

Evidencia: B-02_spa.png; C-01_spa.png

C-01 · Cálculos · Pasa

Precondición: A1 a 12.50

Datos: Cantidad 3

Esperado: 12.50 × 3 = 37.50

Obtenido: Subtotal y total S/ 37.50

Modalidad: SPA + prueba automatizada

Evidencia: C-01_spa.png

Matriz de pruebas - continuación

C-02 · Cálculos · Pasa

Precondición: A2 a 18.90

Datos: Cantidad 3

Esperado: 18.90 × 3 = 56.70

Obtenido: Subtotal y total S/ 56.70, sin cifras adicionales

Modalidad: SPA + prueba automatizada

Evidencia: C-02_spa.png

C-03 · Cálculos · Pasa

Precondición: Tres líneas A1, A2, A3

Datos: A1×2 + A2×1 + A3×1; quitar A2

Esperado: 25.00 + 18.90 + 1.10 = 45.00; 45.00 − 18.90 = 26.10

Obtenido: Total pasa de S/ 45.00 a S/ 26.10

Modalidad: SPA + prueba automatizada

Evidencia: C-03-antes_spa.png; C-03_spa.png

C-04 · Cálculos · Pasa

Precondición: Detalle final C-03

Datos: Registrar A1×2 y A3×1

Esperado: 12.50 × 2 + 1.10 = 26.10; 201

Obtenido: API 201 con 26.10; venta visual 000004 también S/ 26.10

Modalidad: SPA + API HTTP real

Evidencia: C-04_spa.png; C-04_api.json

Matriz de pruebas - continuación

V-01 · Validaciones · Pasa

Precondición: Formulario recién abierto

Datos: Sin cliente ni líneas

Esperado: Mensajes; no aparece resumen ni POST

Obtenido: Mensajes de cliente/detalle y resumen cerrado

Modalidad: SPA + prueba automatizada

Evidencia: V-01_spa.png; frontend-tests.txt

V-02 · Validaciones · Pasa

Precondición: Una línea elegida

Datos: Cantidad 0 y luego 2.5

Esperado: Ambas inválidas; API 400 sin descontar stock

Obtenido: SPA rechaza ambas; API decimal inicialmente 201, corregida a 400

Modalidad: SPA + prueba automatizada

Evidencia: V-02-cero_spa.png; V-02_spa.png; API-V02-2.5_api.json; hallazgo-H01-antes.json

V-03 · Validaciones · Pasa

Precondición: A4 stock 2

Datos: Cantidad 3

Esperado: Supera stock 2; no avanza; API 409

Obtenido: Aviso Supera el stock (2); HTTP 409

Modalidad: SPA + prueba automatizada

Evidencia: V-03_spa.png; API-V03_api.json

Matriz de pruebas - continuación

V-04 · Validaciones · Pasa

Precondición: A1 ya seleccionado

Datos: Agregarlo otra vez

Esperado: Cantidad +1; una sola línea

Obtenido: En la venta: 2; al agregar queda 3 sin duplicar

Modalidad: SPA + prueba automatizada

Evidencia: V-04_spa.png; frontend-tests.txt

V-05 · Validaciones · Pasa

Precondición: Cliente/producto activos

Datos: Mismo producto en dos líneas

Esperado: Regla: rechazar con 409 sin descontar

Obtenido: API HTTP 409 por producto repetido

Modalidad: API HTTP real; colección Postman preparada

Evidencia: V-05_api.json

F-01 · Confirmación · Pasa

Precondición: Resumen abierto

Datos: Volver

Esperado: Cerrar resumen sin POST; conservar líneas

Obtenido: Resumen cerrado; dos líneas y S/ 26.10 conservados; prueba verifica cero POST

Modalidad: SPA + prueba automatizada

Evidencia: F-01-antes_spa.png; F-01_spa.png; frontend-tests.txt

Matriz de pruebas - continuación

F-02 · Confirmación · Pasa

Precondición: Formulario válido revisado

Datos: Invocar Confirmar dos veces consecutivas

Esperado: Una sola petición POST

Obtenido: HttpTestingController recibe exactamente un POST

Modalidad: Prueba automatizada de componente

Evidencia: frontend-tests.txt

M-01 · Mensajes · Pasa

Precondición: Dos pestañas con A4 stock 2

Datos: Cada caja solicita cantidad 2

Esperado: Primera 201; segunda 409; puede corregir

Obtenido: API concurrente [201,409]; SPA conserva detalle y actualiza a stock 0

Modalidad: Dos pestañas SPA + API concurrente real

Evidencia: M-01-primera_spa.png; M-01_spa.png; M-01_api.json

M-02 · Mensajes · Pasa

Precondición: Cliente 81000003 inactivo

Datos: Venta con una línea válida

Esperado: 409: no se puede vender al cliente inactivo

Obtenido: HTTP 409 con mensaje de cliente inactivo

Modalidad: API HTTP real; colección Postman preparada

Evidencia: M-02_api.json

Matriz de pruebas - continuación

I-01 · Impresión · Falla

Precondición: Comprobante y anulada existentes

Datos: Imprimir al registrar, anulada y consulta

Esperado: DNI y pie, sello, filtros; guardar los tres como PDF

Obtenido: Tres capturas previas acreditan presentación; faltan los tres archivos PDF nativos

Modalidad: Inspección de capturas anteriores; entrega de archivos incompleta

Evidencia: I-01-comprobante_spa.png; I-01-anulada_spa.png; I-01-consulta_spa.png

X-01 · Consistencia · Pasa

Precondición: Venta C-04 registrada

Datos: Productos y consulta

Esperado: Stock baja 2; total 26.10 y 2 ítems

Obtenido: API: 40−2=38 en primera venta; SPA después de repetir C-04: 38−2=36. Comprobante 000004: 26.10 y 2 productos

Modalidad: SPA + API HTTP real

Evidencia: X-01_spa.png; C-04_spa.png; X-01_api.json

P-01 · Buscadores · Pasa

Precondición: Formulario abierto

Datos: Texto zzz

Esperado: Aviso sin coincidencias

Obtenido: Ningún cliente activo coincide

Modalidad: SPA + prueba automatizada

Evidencia: P-01_spa.png

Matriz de pruebas - continuación

P-02 · Validaciones · Pasa

Precondición: Cliente y A4 elegidos

Datos: Cantidad −1

Esperado: Rechazar; no confirmar

Obtenido: Cantidad entera mayor que cero; resumen no aparece

Modalidad: SPA + prueba automatizada

Evidencia: P-02_spa.png; frontend-tests.txt

P-03 · Confirmación · Pasa

Precondición: Datos válidos sin revisar

Datos: Invocar confirmar directamente

Esperado: No POST

Obtenido: Prueba verifica ninguna petición /ventas

Modalidad: Prueba automatizada de componente

Evidencia: frontend-tests.txt

P-04 · Mensajes · Pasa

Precondición: Cliente activo

Datos: Producto inexistente 99999999

Esperado: HTTP 404 con mensaje

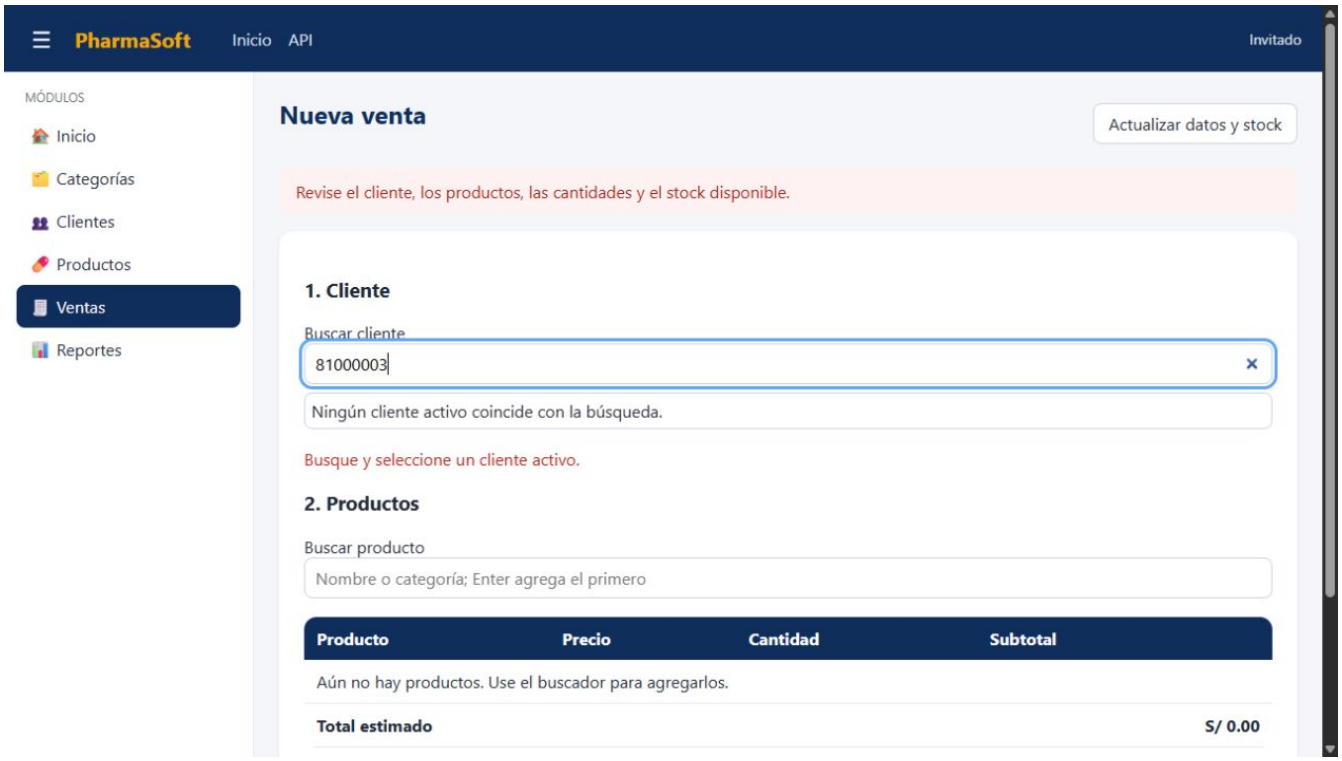
Obtenido: HTTP 404; no crea venta

Modalidad: API HTTP real

Evidencia: P-04_api.json

Evidencia B-01

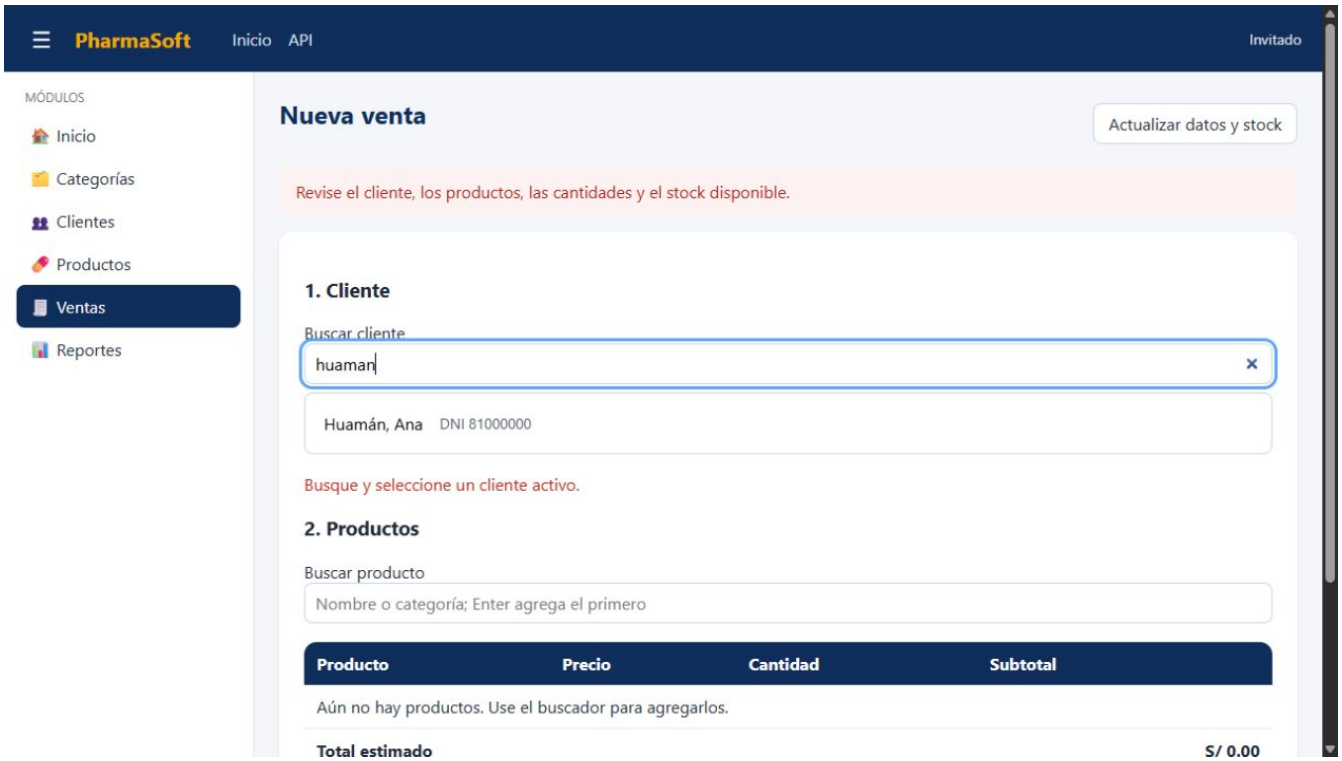
DNI inactivo 81000003: no aparece.



B-01-inactivo_spa.png

Evidencia B-01

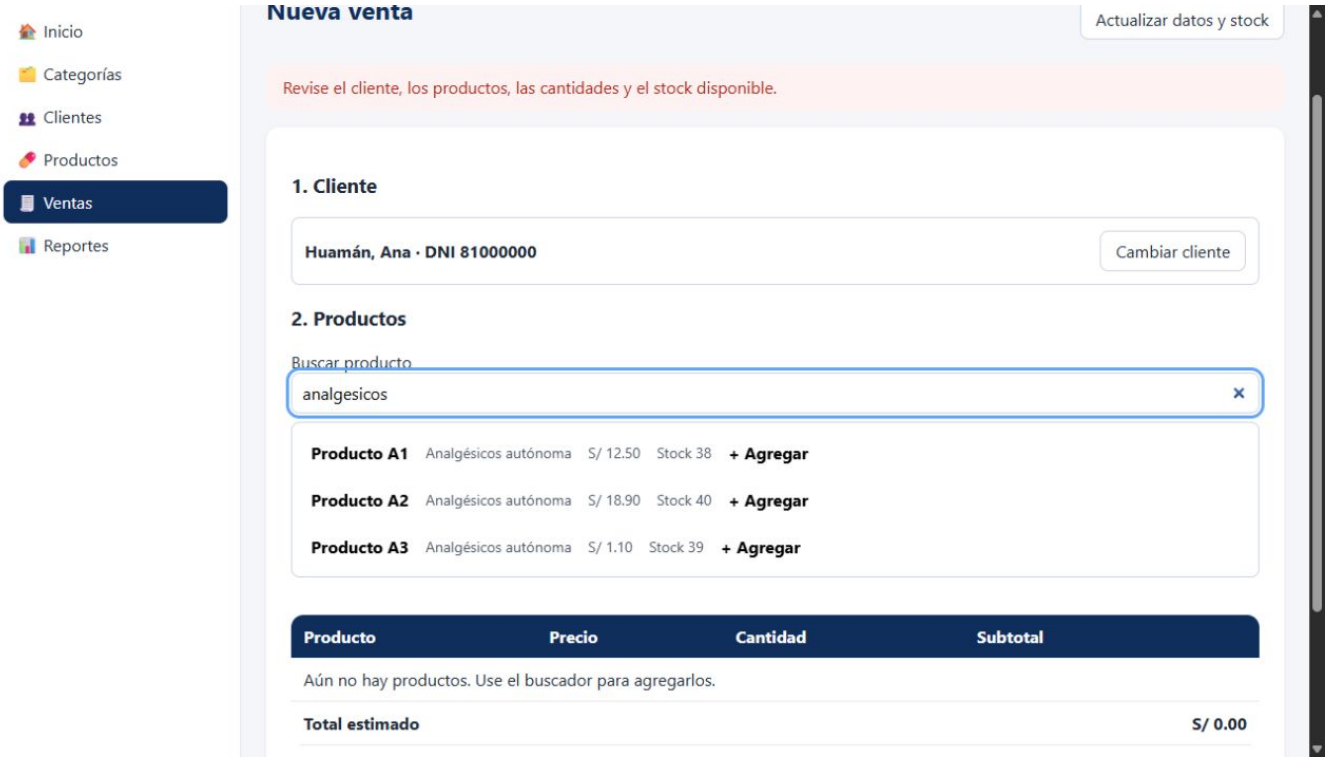
huaman encuentra a Ana Huamán.



B-01_spa.png

Evidencia B-02

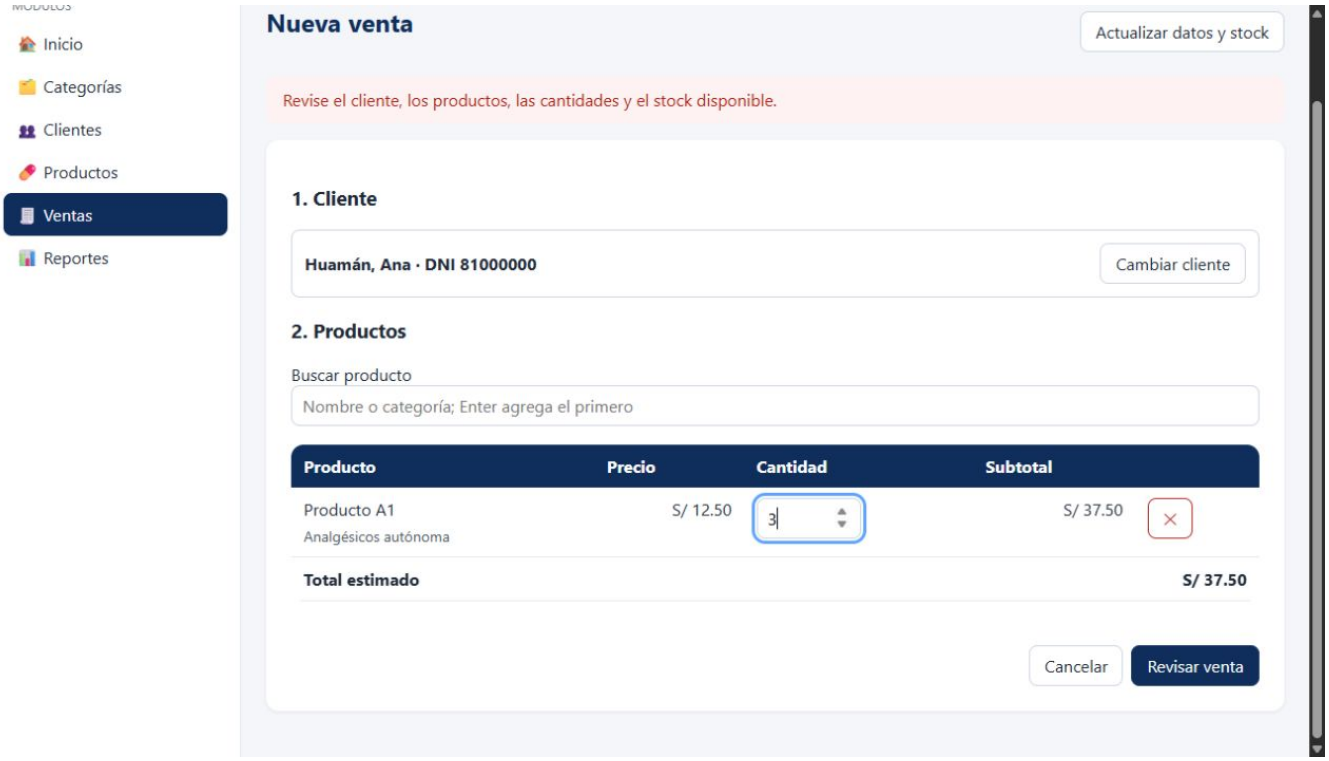
Búsqueda por categoría; precios y stock visibles.



B-02_spa.png

Evidencia C-01

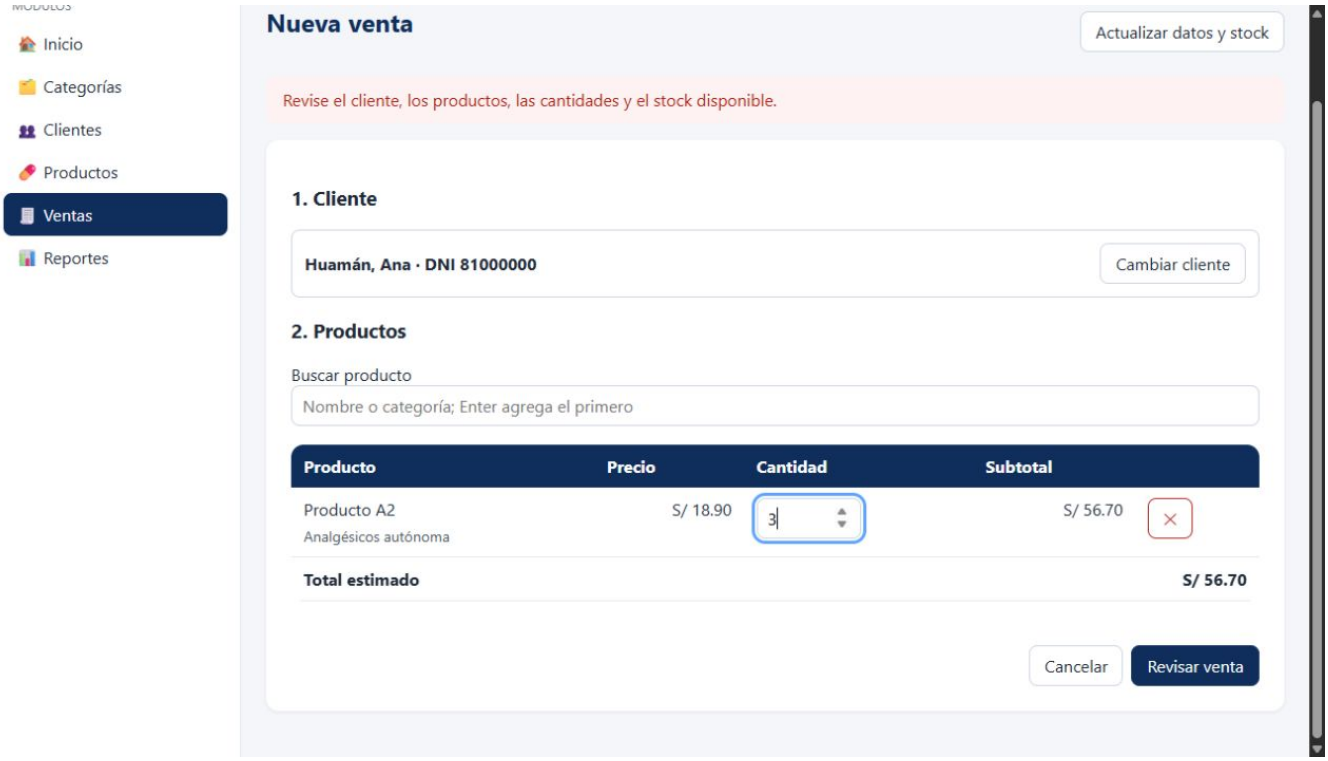
12.50 × 3 = 37.50.



C-01_spa.png

Evidencia C-02

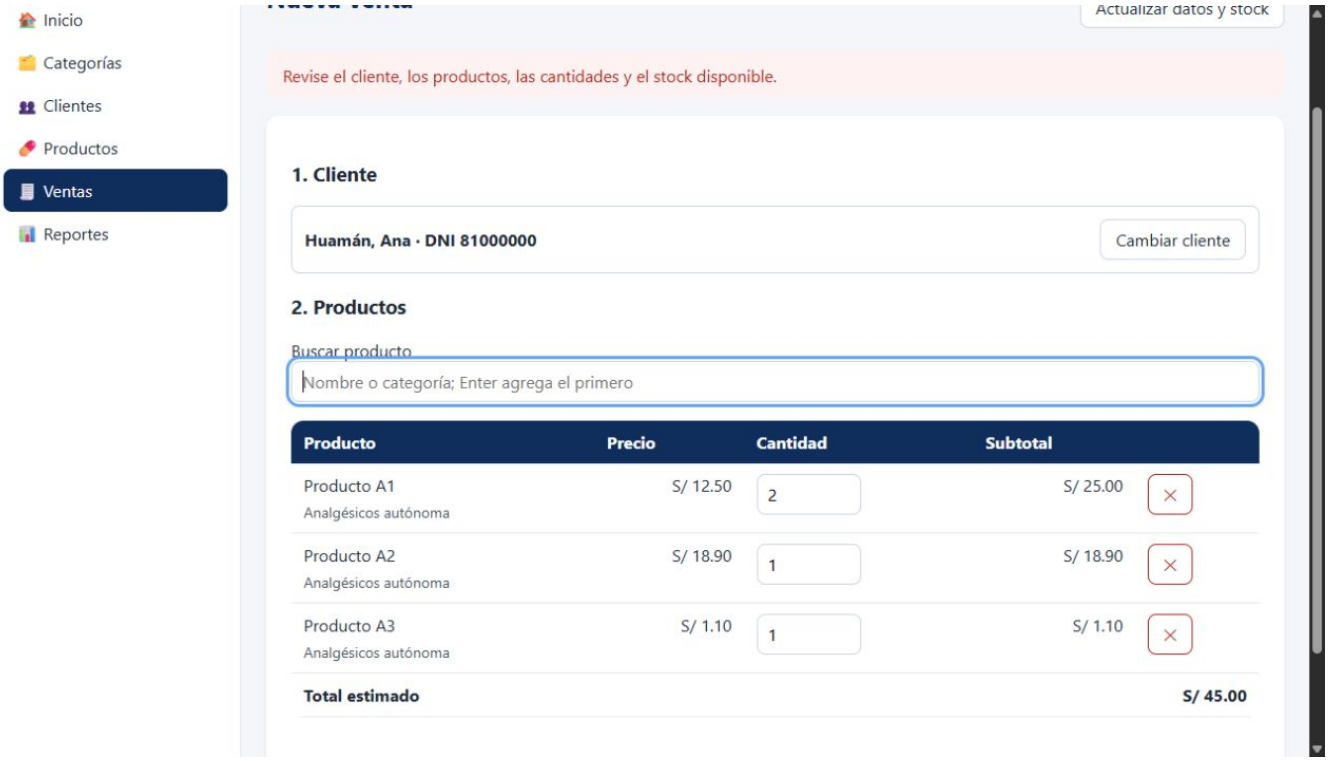
18.90 × 3 = 56.70.



C-02_spa.png

Evidencia C-03

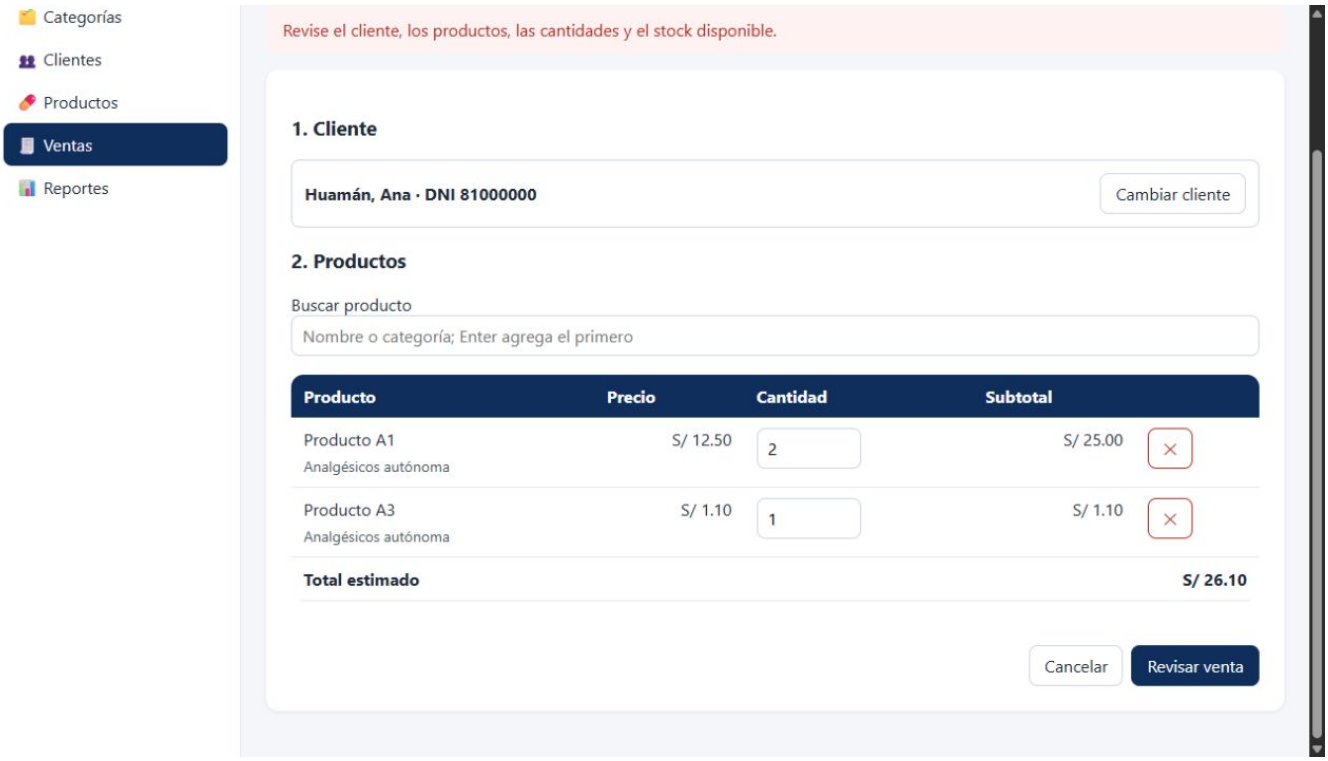
12.50 × 2 + 18.90 + 1.10 = 45.00.



C-03-antes_spa.png

Evidencia C-03

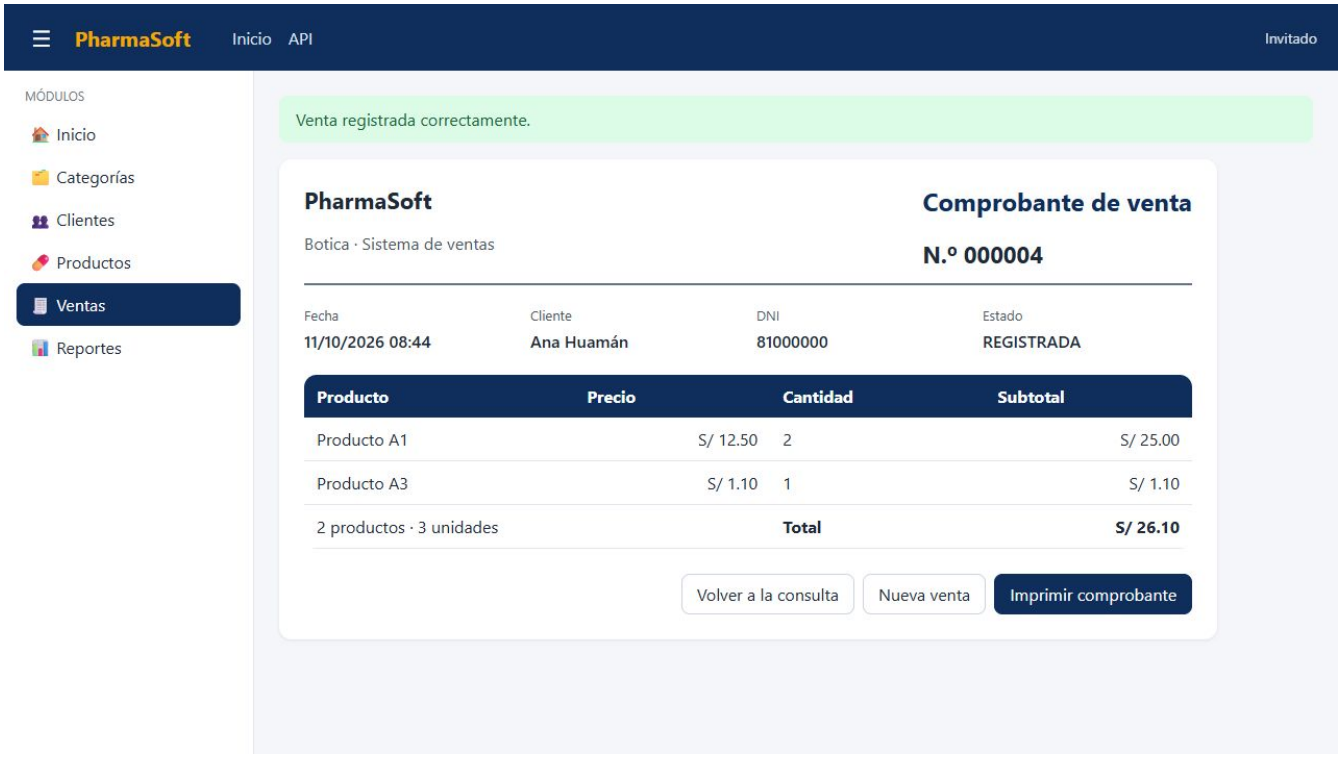
45.00 − 18.90 = 26.10.



C-03_spa.png

Evidencia C-04

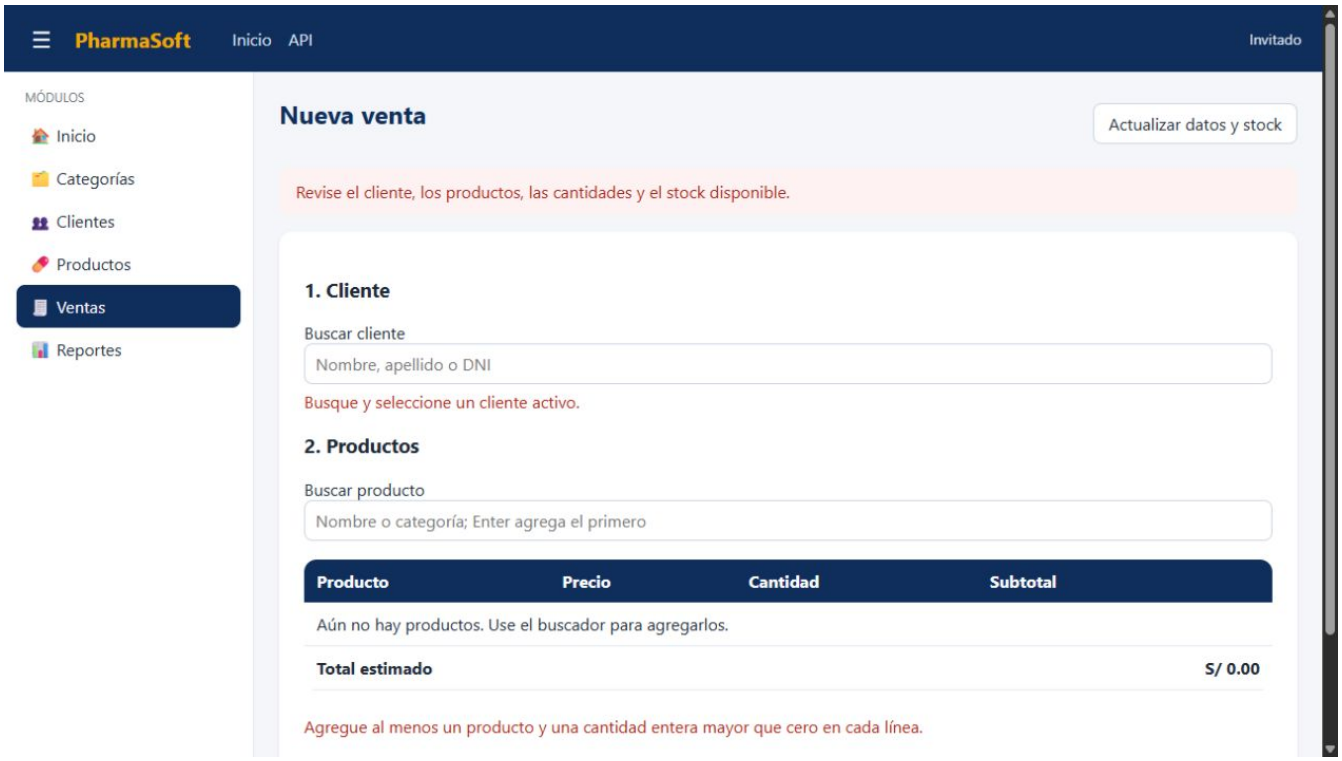
Comprobante 000004: total definitivo S/ 26.10 y dos productos.



C-04_spa.png

Evidencia V-01

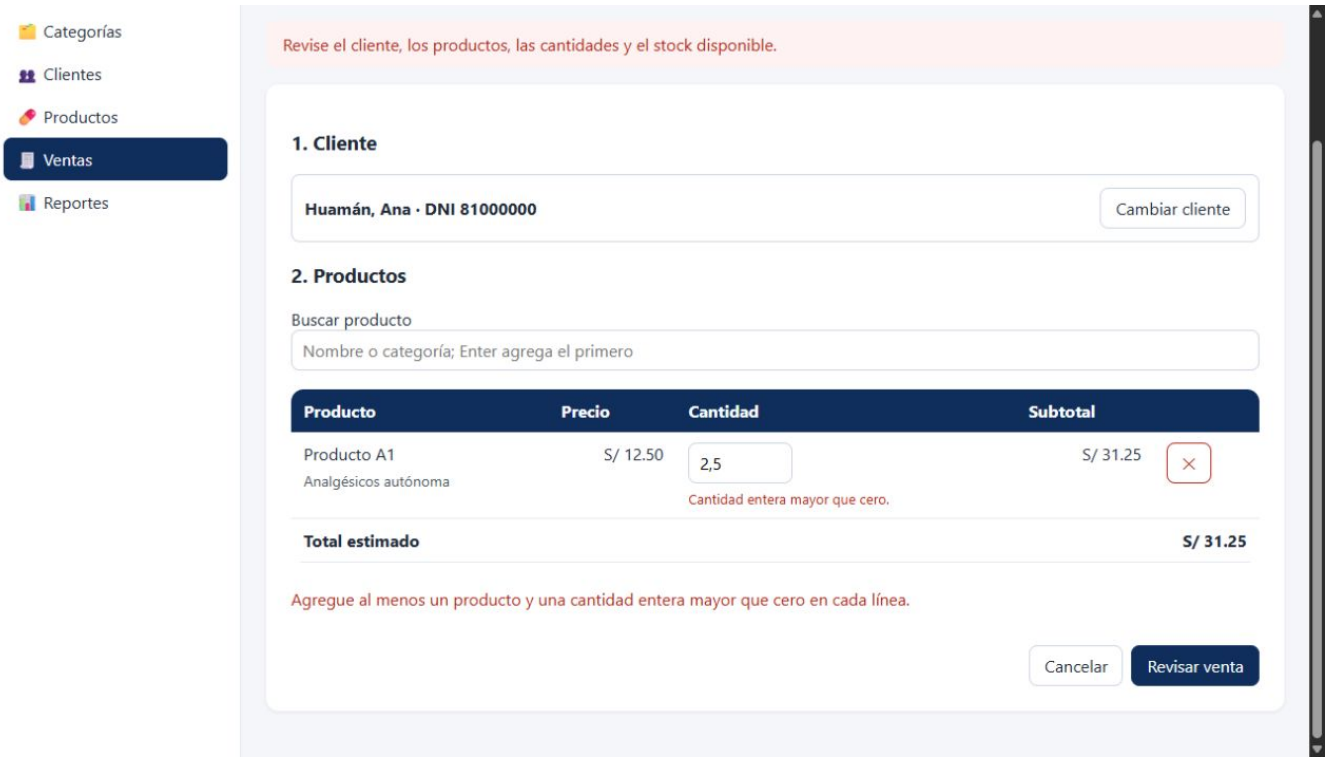
Formulario vacío: mensajes de cabecera y detalle.



V-01_spa.png

Evidencia V-02

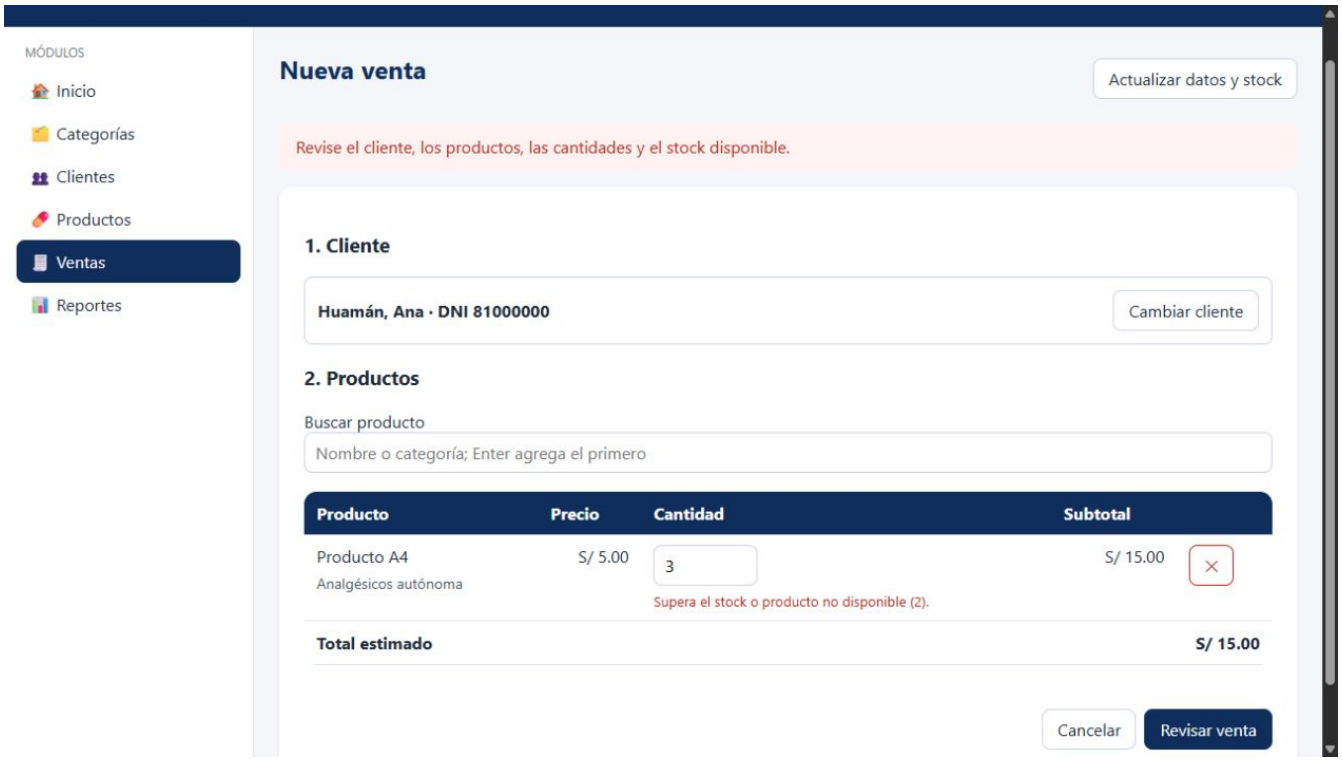
Cantidad fraccionaria rechazada por la SPA.



V-02_spa.png

Evidencia V-03

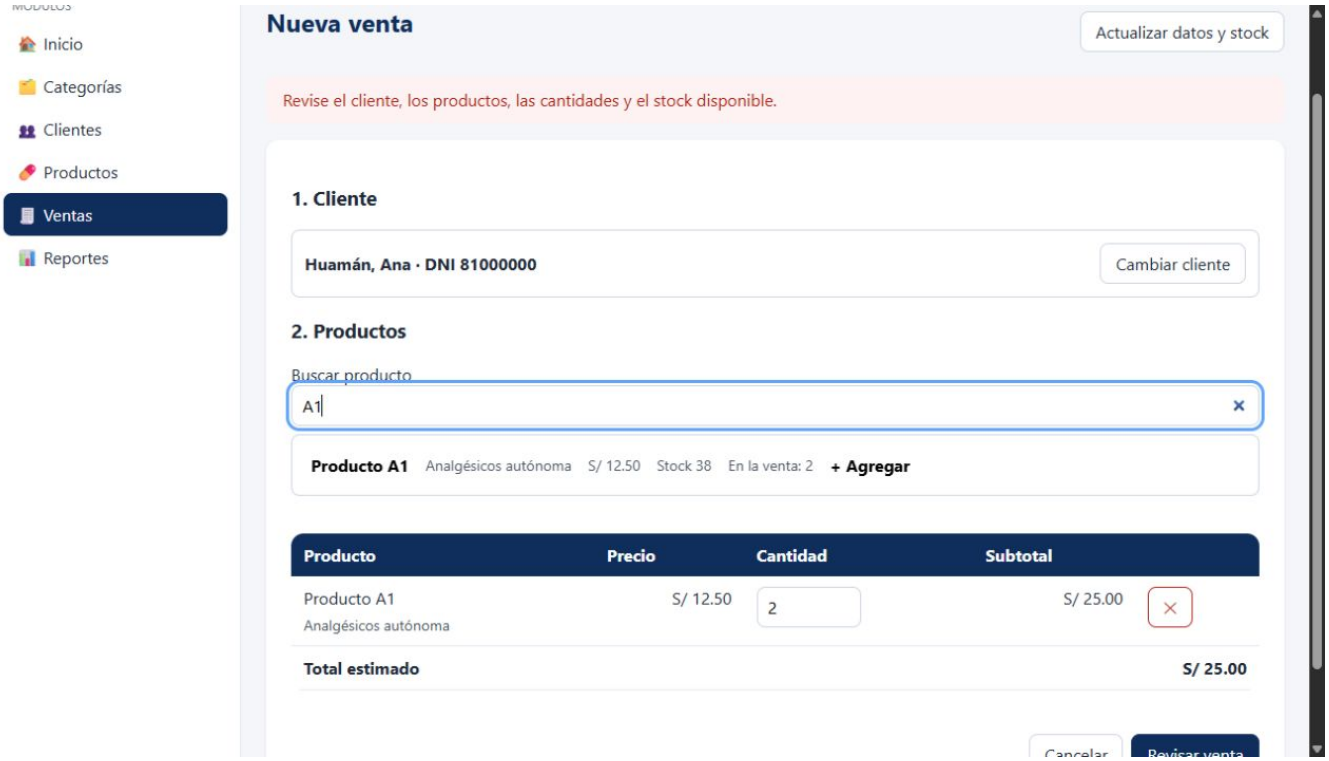
Cantidad 3 supera stock 2.



V-03_spa.png

Evidencia V-04

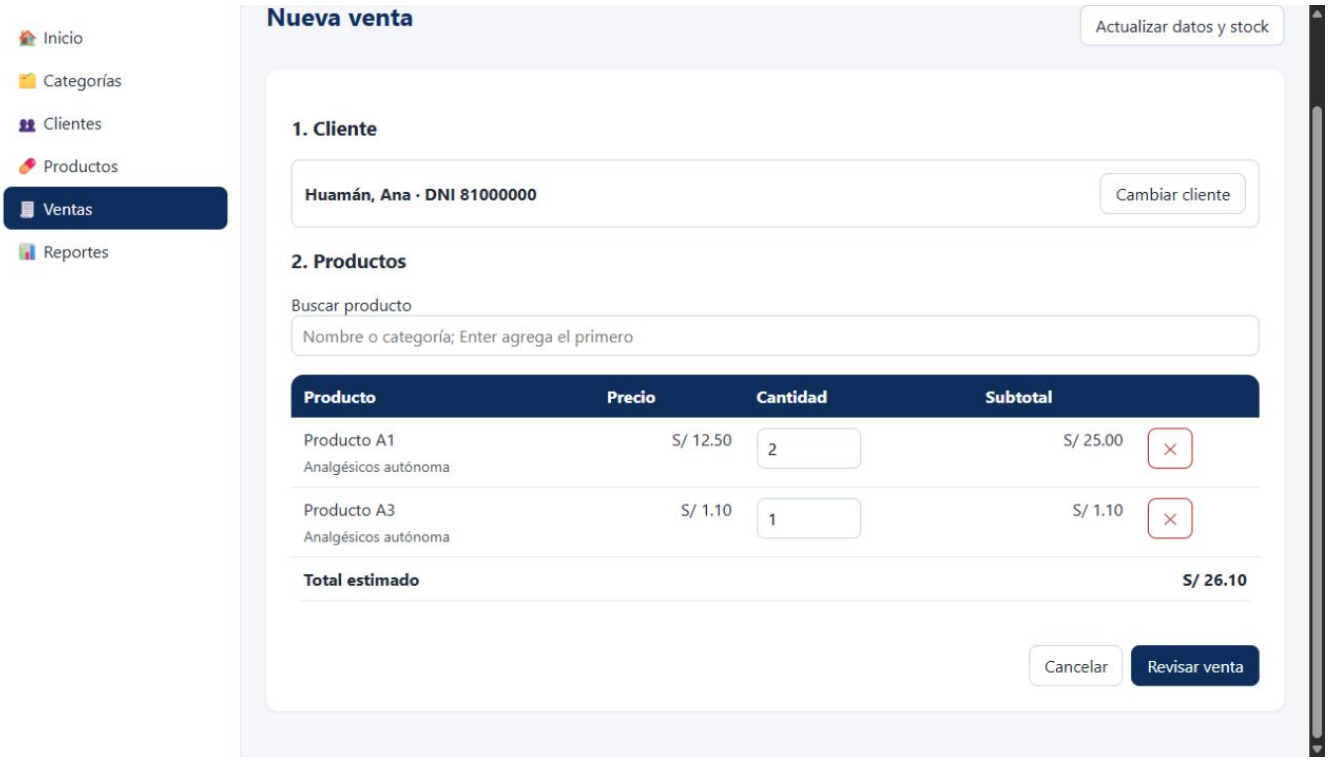
Producto identificado como En la venta: 2.



V-04_spa.png

Evidencia F-01

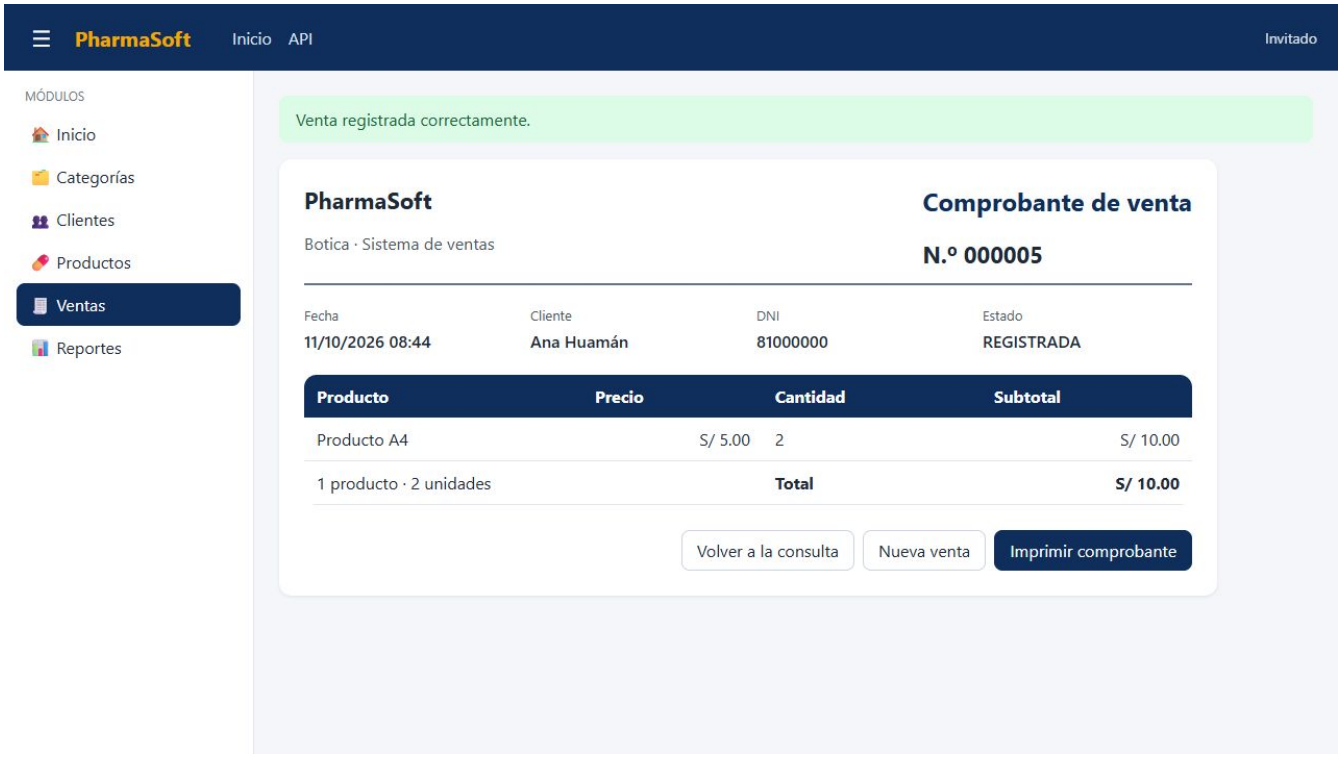
Volver conserva el detalle y cierra el resumen.



F-01_spa.png

Evidencia M-01

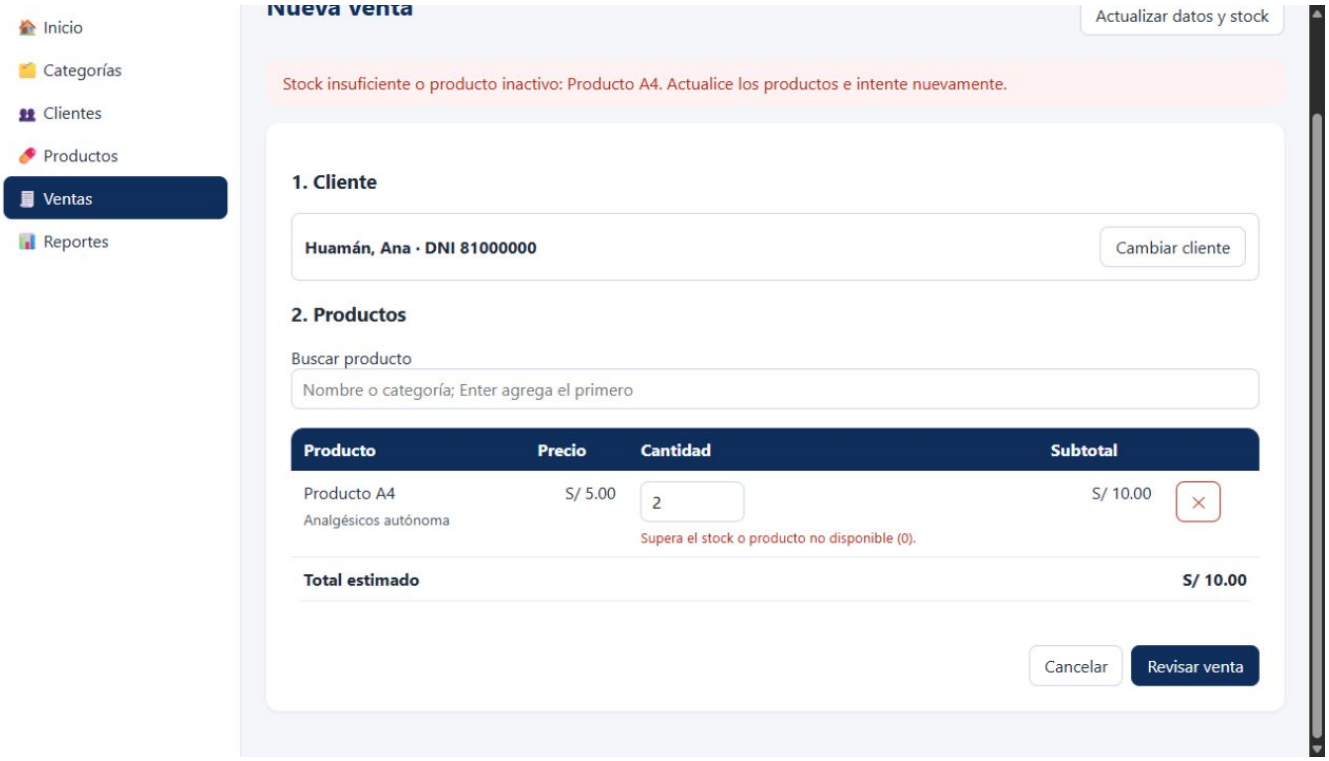
Primera caja: 2 × 5.00 = 10.00.



M-01-primera_spa.png

Evidencia M-01

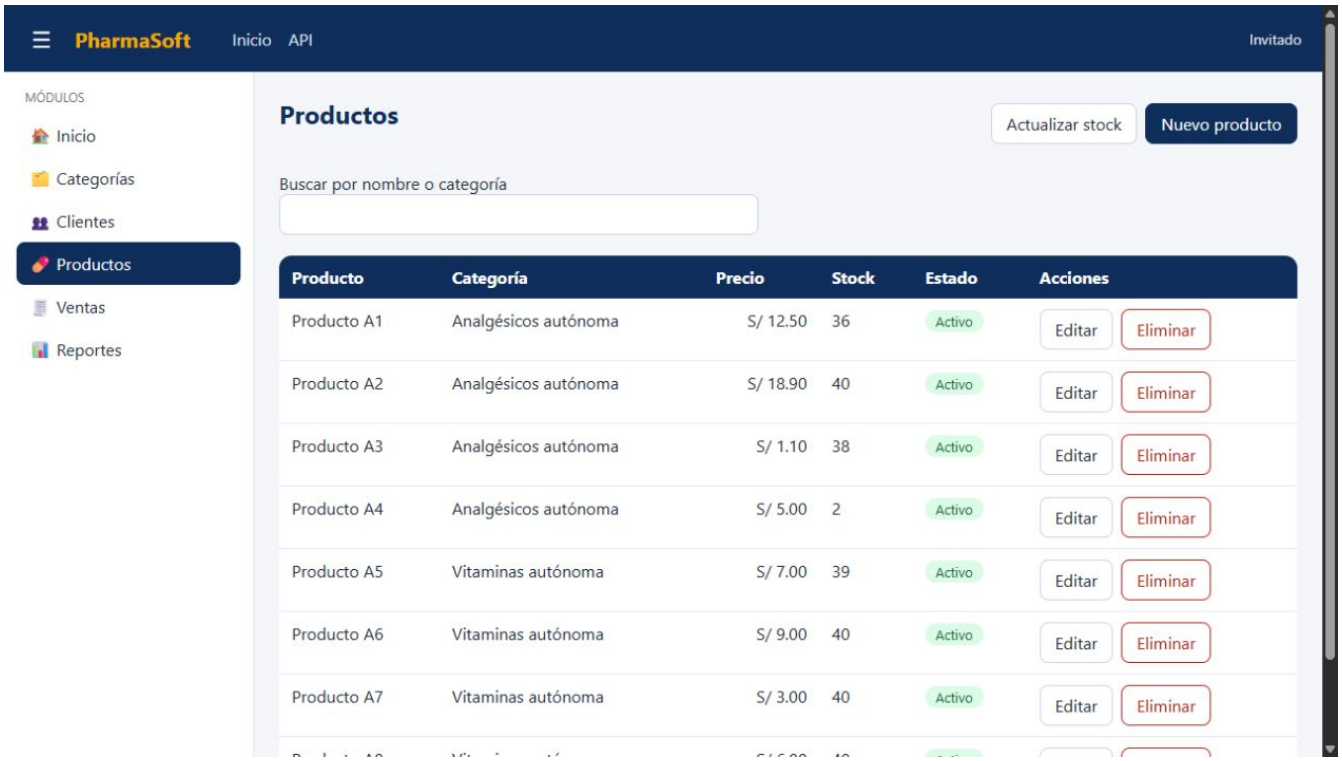
Segunda caja: conflicto, detalle conservado y stock 0.



M-01_spa.png

Evidencia X-01

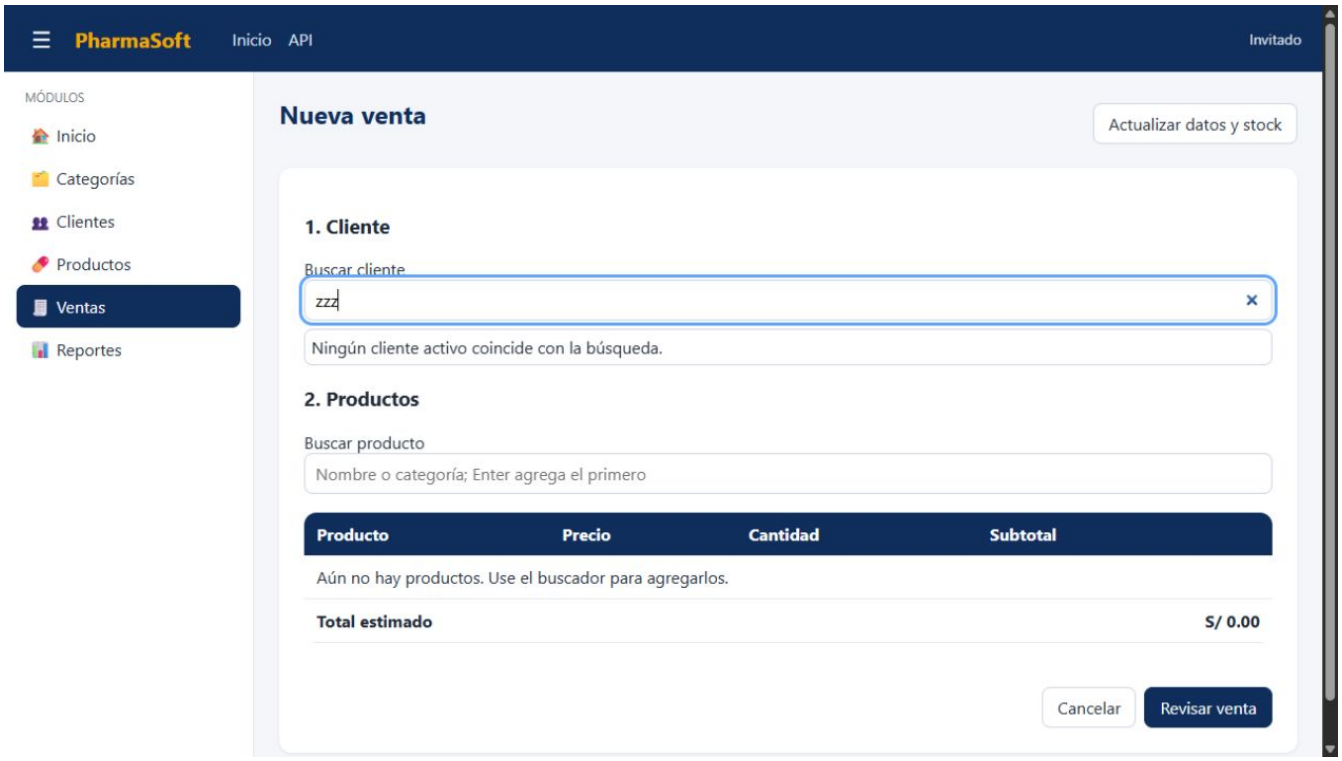
Stock después de repetir la venta C-04: A1=36 y A3=38.



X-01_spa.png

Evidencia P-01

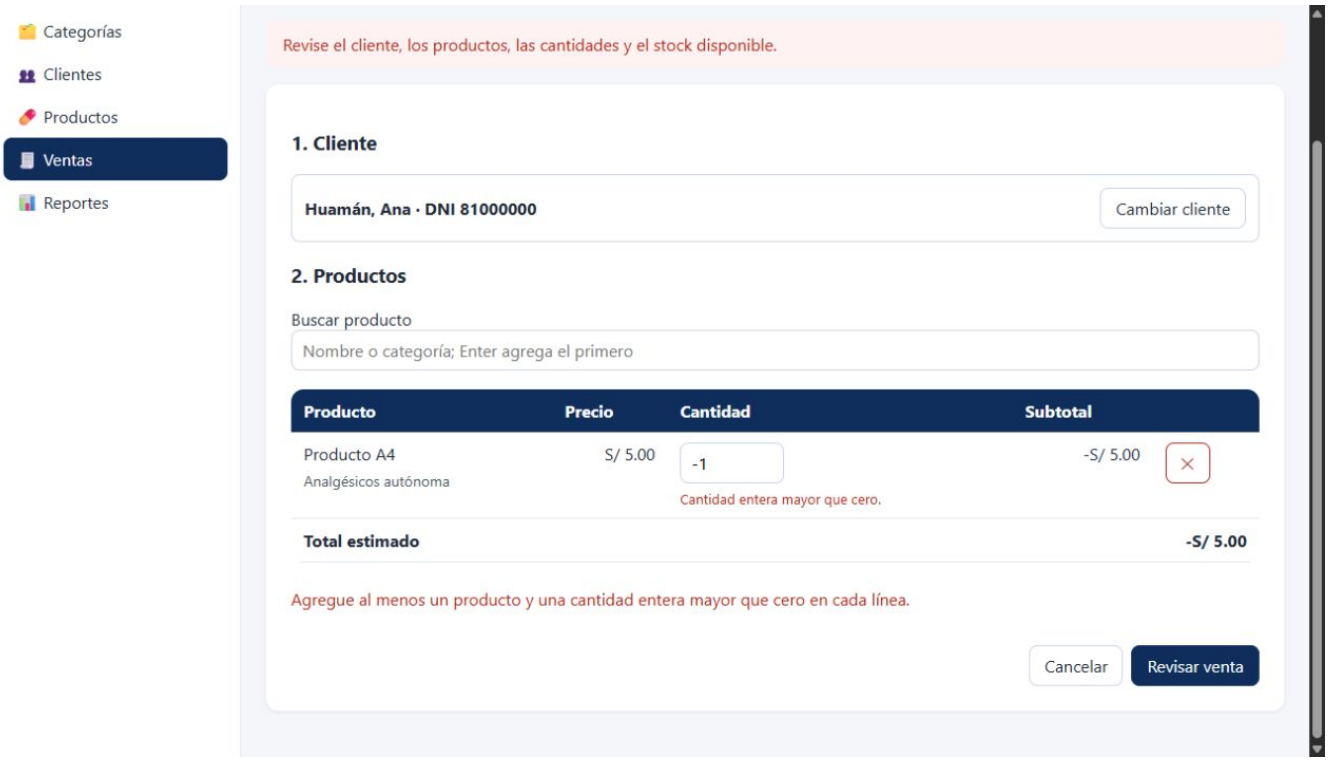
Texto inexistente muestra el aviso.



P-01_spa.png

Evidencia P-02

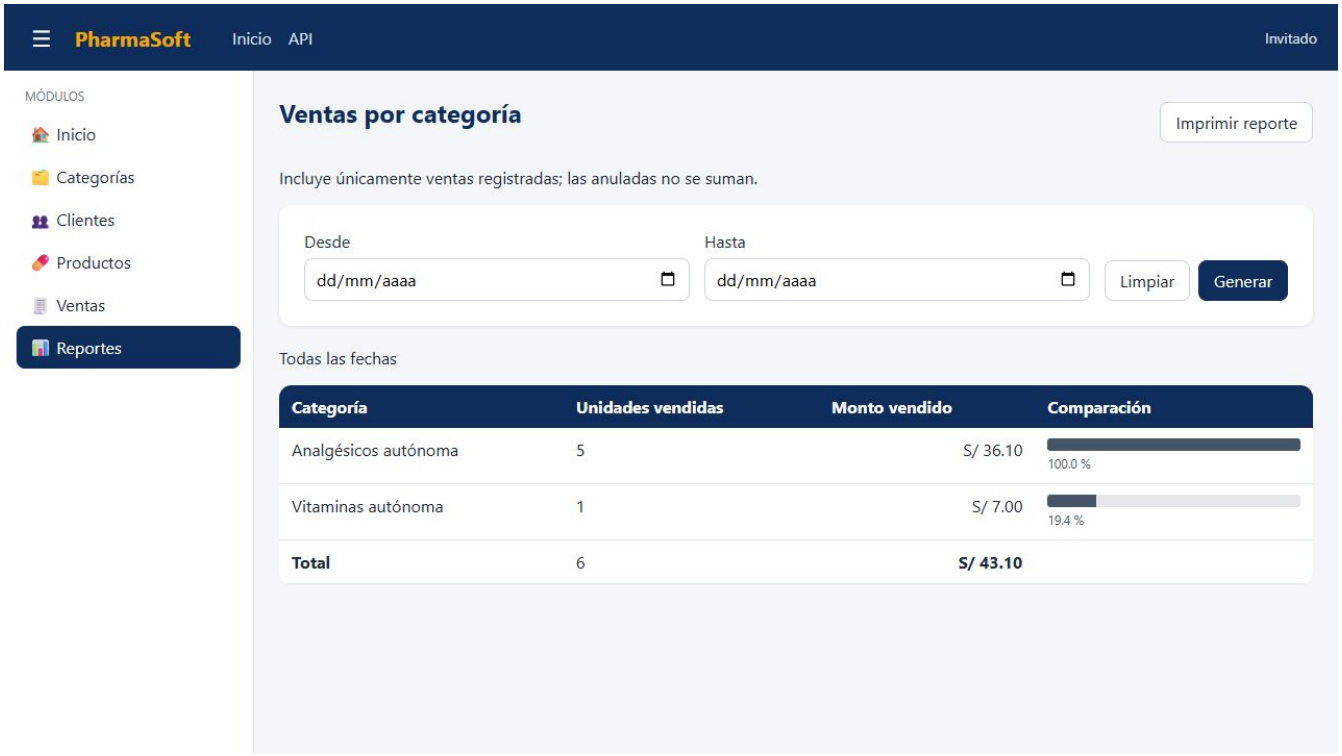
Cantidad negativa no permite confirmar.



P-02_spa.png

Evidencia R-01

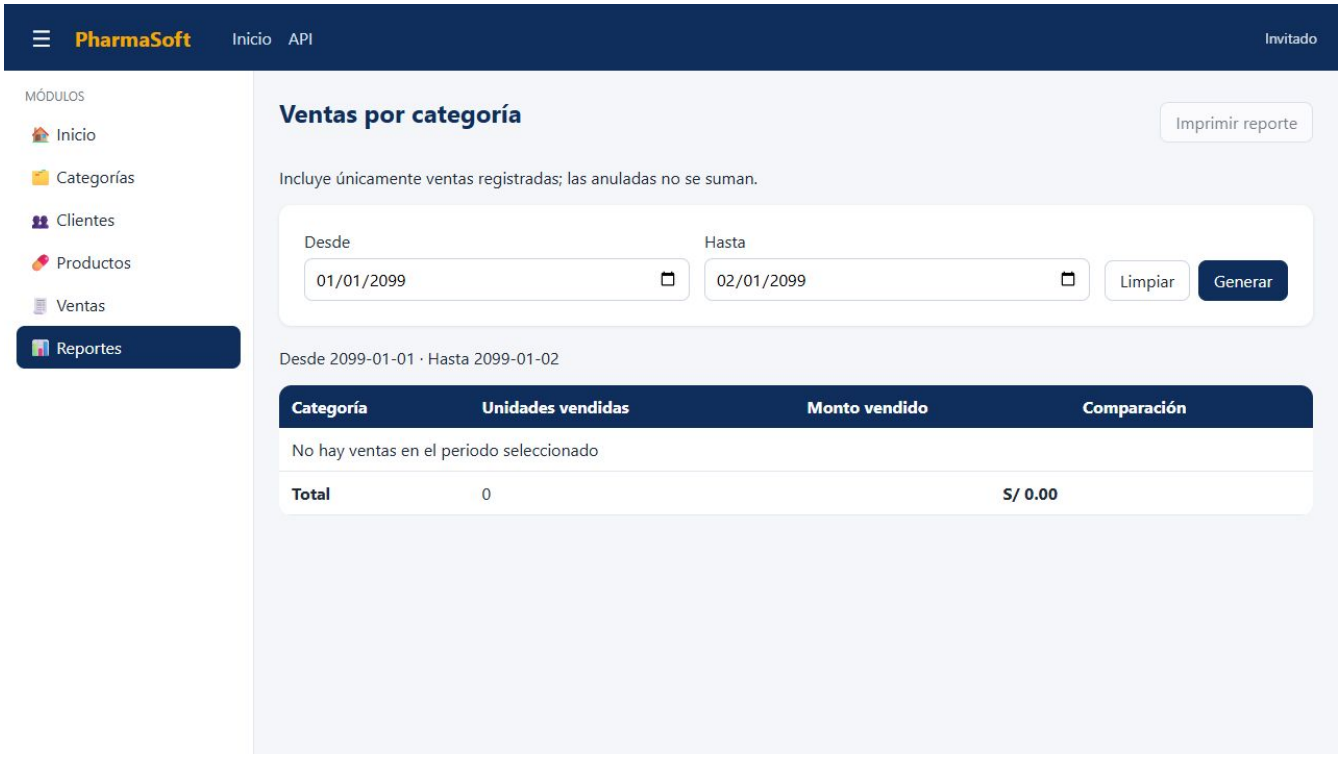
Comparación: 36.10/36.10=100 % y 7.00/36.10=19.4 %. Total 43.10; 6 unidades. Captura previa a las ventas visuales posteriores.



R-01_spa.png

Evidencia R-06

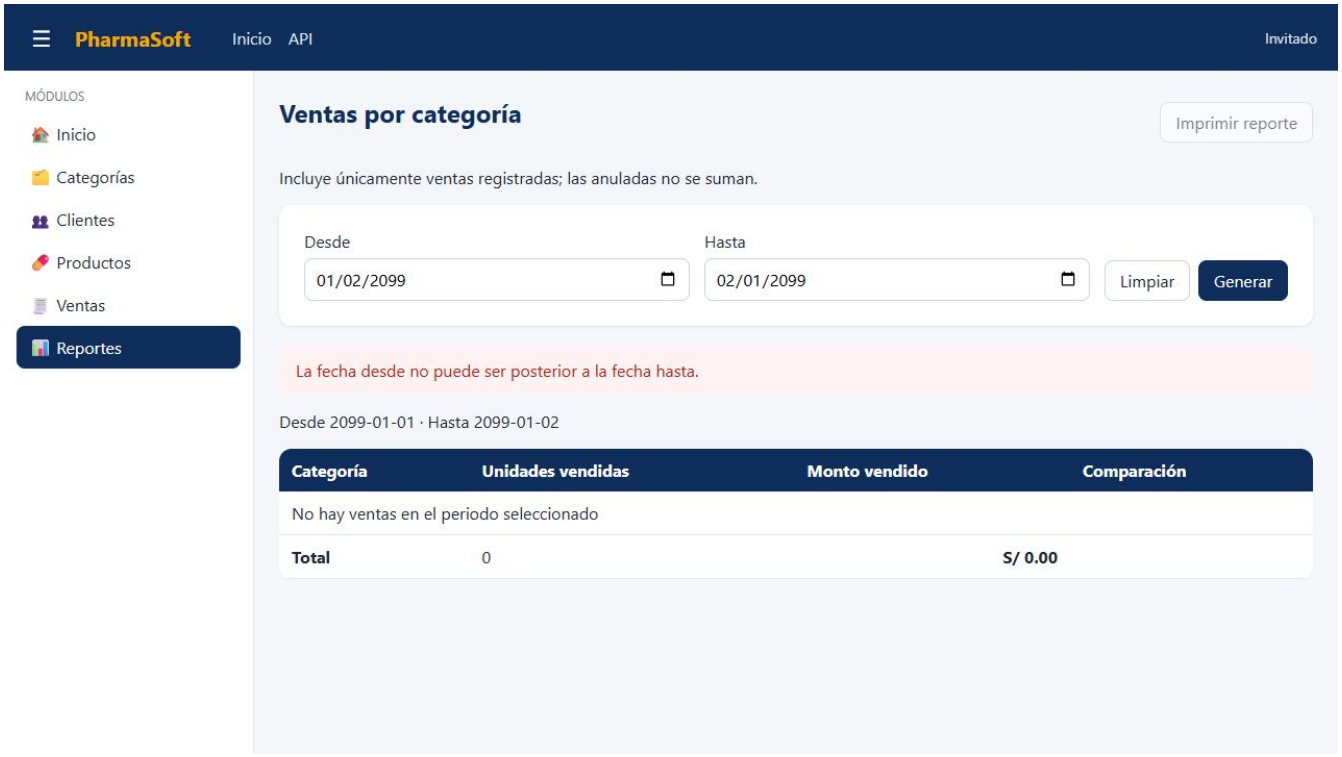
Fechas futuras: No hay ventas en el periodo seleccionado.



R-06_spa.png

Evidencia R-02

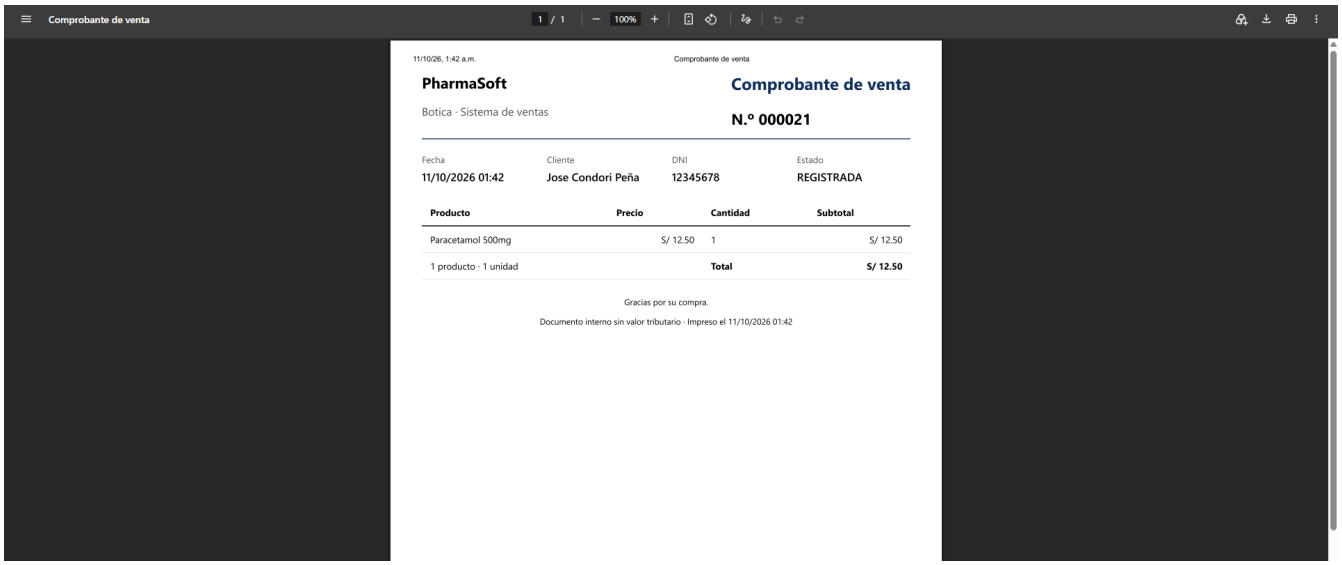
Desde posterior a Hasta: validación de la SPA.



R-02_spa.png

Evidencia I-01

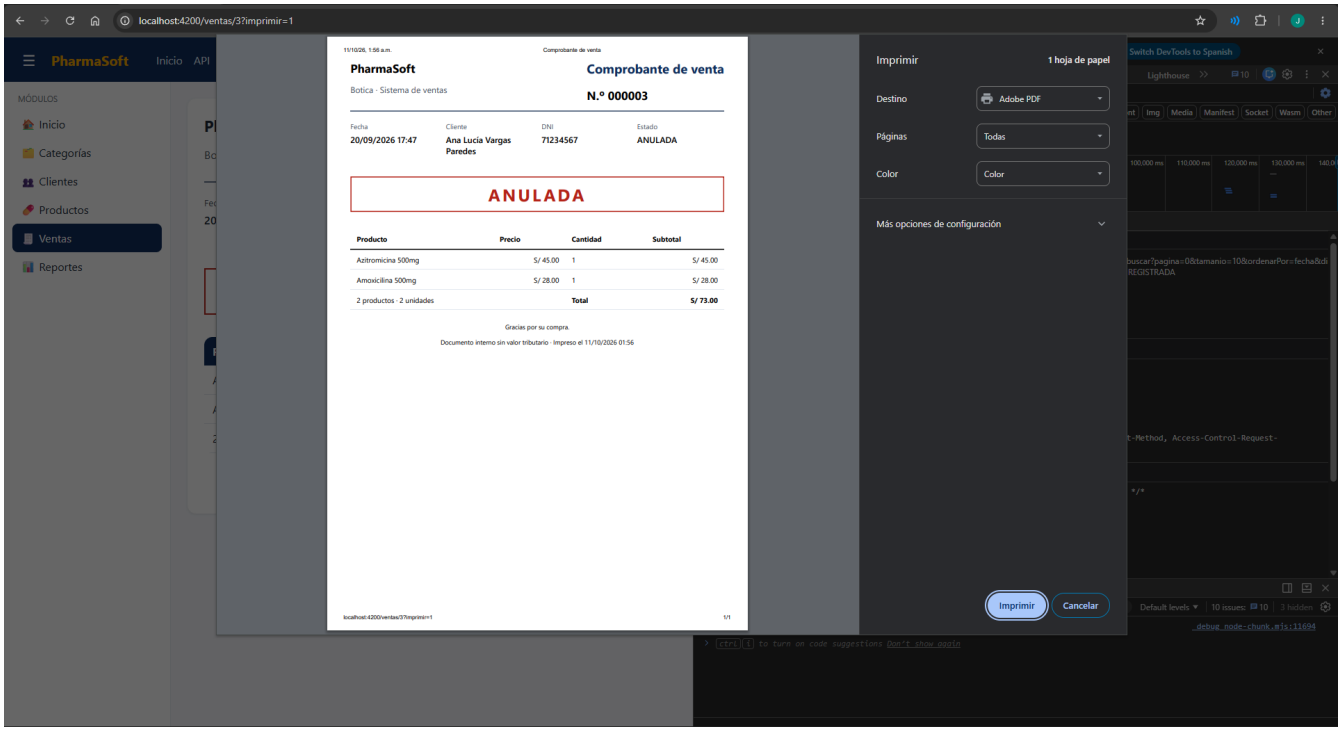
Captura aportada previamente por el estudiante: comprobante impreso, con DNI y pie.



I-01-comprobante_spa.png

Evidencia I-01

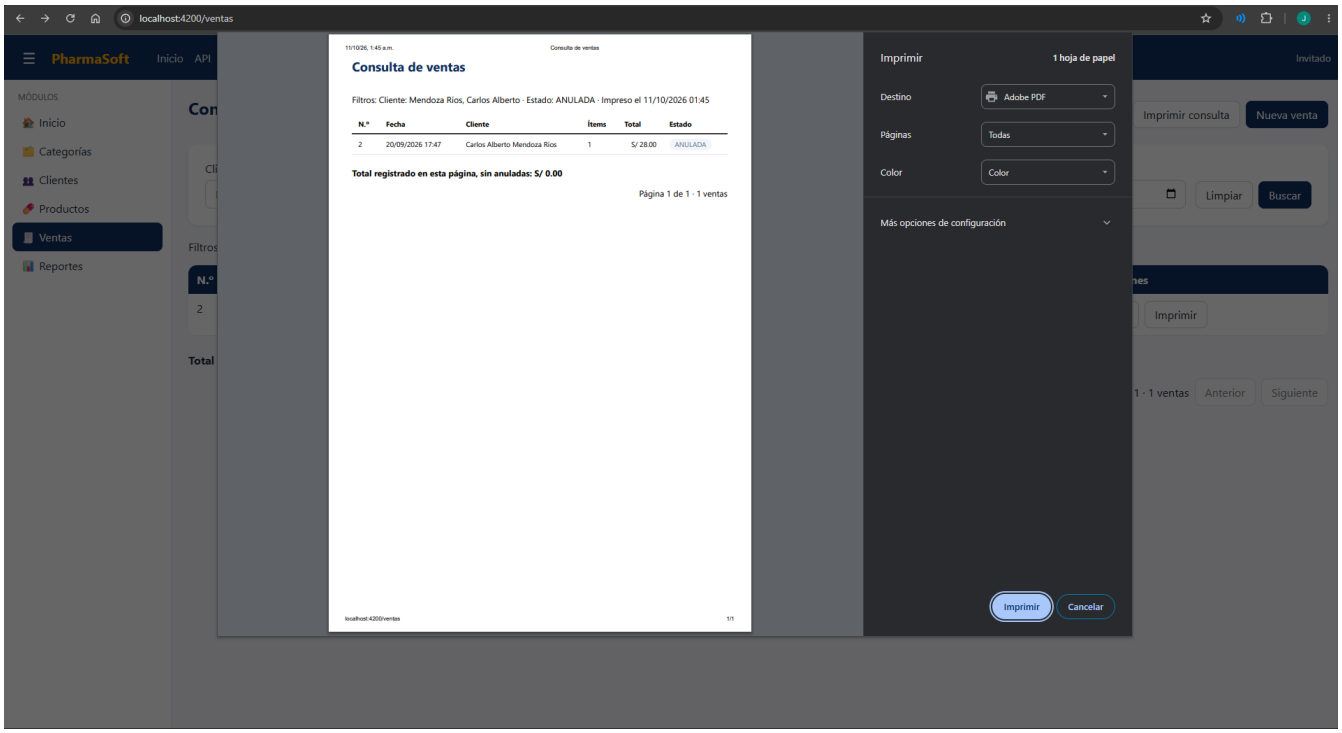
Captura previa del estudiante: sello ANULADA en vista de impresión.



I-01-anulada_spa.png

Evidencia I-01

Captura previa del estudiante: filtros y total sin anuladas. Faltan los archivos PDF nativos.



I-01-consulta_spa.png

Evidencias HTTP y regresión

Caso	Resultado observado	Estado
C-04	HTTP 201: 26.1	Pasa
V-05	HTTP 409: Un producto no puede repetirse en el detalle	Pasa
M-02	HTTP 409: No se puede registrar una venta para un cliente inactivo	Pasa
API-V01	HTTP 400: Existen errores de validacion	Pasa
API-V02-0	HTTP 400: Existen errores de validacion	Pasa
API-V02-2.5	HTTP 400: None	Pasa
API-V03	HTTP 409: Stock insuficiente o producto inactivo: Producto A4. Actualice los productos e intente nuevamente.	Pasa
M-01	HTTP [201, 409]; stock 0	Pasa
X-01	stock A1=38; total=26.1; lineas=2	Pasa
P-04	HTTP 404: Producto no encontrado con id: 99999999	Pasa
R-01	HTTP 200; total 36.1; 1 categorías	Pasa
R-06	HTTP 200: []	Pasa
R-07	HTTP 409: {'timestamp': '2026-10-11T08:39:51.0692521', 'status': 409, 'error': 'Conflict', 'message': "El rango de fechas es inválido: 'desde' (2026-10-11) es posterior a 'hasta' (2026-10-01)", 'path': '/api/v1/reportes/ventas-por-categoria', 'validationErrors': None}	Pasa

Las respuestas completas, cuerpos enviados y métodos están en docs/evidencias-s9/*_api.json. Las peticiones se ejecutaron mediante un cliente HTTP Python contra el backend real de H2. La colección Postman se entrega para reproducción; no se presenta esta tabla como una captura de Postman.

Entrega local y pasos pendientes

Archivos preparados: docs/matriz-pruebas-s9.md, docs/informe-s9.pdf, docs/evidencias-s9/ y postman/PharmaSoft-S9-Ventas.json. El reporte usa models/reporte.model.ts, services/reporte-service.ts, pages/ventas-categoria/ y reportes.routes.ts. La opción Reportes abre /reportes/ventas-por-categoria mediante carga diferida.

El servicio utiliza categoriaNombre y cantidadVendida en el frontend y adapta los nombres heredados nombreCategoria/cantidadTotal de tarea05. Solo envía fechas con valor; Generar valida el rango. La tabla incorpora CurrencyPipe, barras proporcionales y totales computed. Carga, error y vacío tienen comprobación de componente; las capturas nuevas incluyen vacío y rango inválido.

Para cumplir estrictamente la evidencia pedida: conservar los tres PDF nativos de I-01, completar las capturas de F12 para cada caso con petición y capturar V-05/M-02 en Postman. F-02/P-03 cuentan por ahora con regresión automatizada y necesitan su evidencia visual. Estos pendientes no se ocultan en la matriz.

Por indicación del estudiante no se publica nada en GitHub ni se crean commits de esta actividad. En el nuevo repositorio deberá partir de develop con la práctica incorporada, crear feature/pruebas-reporte-ventas-condori y hacer al menos tres commits en días reales distintos. Después corresponde abrir la solicitud hacia develop. No se han alterado fechas de commits.

Ejecución habitual: iniciar tarea05 en IntelliJ y Oracle; ejecutar npm.cmd start en frontend. Abrir localhost:4200/reportes/ventas-por-categoria. Reiniciar el backend para aplicar el rechazo de decimales. La colección prepara datos sintéticos: ejecutarla solo en una base vacía de pruebas. El ZIP local excluye historial Git, dependencias y compilados.